FINAL YEAR SOFTWARE ENGINEERING PROJECT

EthioEqub Digital Management System

Project Requirements and Technical Design Document (PRTDD)

A comprehensive software architecture, financial integrity framework, security model, and implementation blueprint for modernizing traditional Ethiopian rotating savings and credit associations (ROSCA / ዕቁብ).

DOCUMENT VERSION
Version 1.1 (Approved Baseline Specification)

REVIEW STATUS
Requirements Approved & Baseline Finalized (Phase 2 Sign-Off)

AUTHOR / ROLE
Software Engineering Student Developer

TECHNICAL MENTOR
Senior Full-Stack Architect & Cybersecurity Specialist

TARGET DOMAIN
Traditional Ethiopian Equb (ዕቁብ) Group Financial Governance

DATE OF PUBLICATION
October 2026

Document Control & Revision History

Version	Date	Author	Summary of Changes
v0.1	October 2026	Student Developer	Initial requirements capture, scope definition, and high-level structure.
v1.0	October 2026	Student Developer & Mentor	Complete PRTDD draft publication: 26 sections, full architectural design, DB schema, API specs, security protocols.
v1.1	October 2026	Student Developer & Mentor	Phase 2 baseline finalized: confirmed English-first i18n architecture, configurable dual approval for small groups (< 5 members), conditional guarantor for early payouts, and email + in-app notification strategy with pluggable SMS adapter.

Stakeholder Sign-Off Register

Stakeholder Role	Name	Status	Sign-Off Date
Academic Supervisor	Faculty Review Board	Pending Defense Review	—
Student Lead Developer	Wagnew A.	Draft Completed	October 2026
Technical Architect Mentor	Senior Architect Pair	Verified Architecture	October 2026
Equb Domain Representative	Community Equb Organizer	Consulted on Rules	October 2026

> **Formal Review Notice**
>
> This document constitutes the baseline specification for the EthioEqub Digital Management System. In accordance with professional engineering standards, no application implementation or folder generation commences until this document is approved. Progress across subsequent phases requires explicit confirmation via the command `next`.

Table of Contents

1. Cover Page & Document Version ..... Page 1

2. Executive Summary ..... Page 3

3. Background and Problem Statement ..... Page 3

4. Project Objectives and Expected Benefits ..... Page 4

5. Project Scope and Limitations ..... Page 5

6. Stakeholders and User Roles ..... Page 6

7. Functional Requirements (FR-001 – FR-020) ..... Page 7

8. Non-Functional Requirements (NFR-001 – NFR-015) ..... Page 9

9. Business Rules (BR-001 – BR-014) ..... Page 11

10. User Journeys and Use Cases (UC-01 – UC-06) ..... Page 13

11. System Architecture and Component Responsibilities ..... Page 15

12. Technology Stack and Justification ..... Page 17

13. Database Design, Entities, Relationships, and ER Diagram ..... Page 19

14. API Design and Representative Endpoints ..... Page 22

15. UI/UX Design Plan and Screen Inventory (S-01 – S-13) ..... Page 25

16. Authentication, Authorization, Privacy, Security, and Audit Requirements ..... Page 28

17. Financial Transaction Integrity, Reconciliation, and Reporting ..... Page 30

18. Notification and Payment Integration Strategy ..... Page 32

19. Validation Rules and Error-Handling Strategy ..... Page 33

20. Testing Strategy and Acceptance Criteria (AC-01 – AC-08) ..... Page 35

21. Development Phases and Milestones (Phases 1 – 12) ..... Page 37

22. Risks, Assumptions, and Mitigation Strategies ..... Page 39

23. Deployment, Backup, Recovery, and Maintenance Plan ..... Page 41

24. Installation Prerequisites and Developer Environment Setup ..... Page 42

25. Final Project Acceptance Checklist ..... Page 43

26. Glossary of Cultural and Technical Terms ..... Page 44

2. Executive Summary

An **Equb** (ዕቁብ) is a traditional Ethiopian Rotating Savings and Credit Association (ROSCA) that serves as one of the most prominent informal financial mechanisms in Ethiopian society. In an Equb, a group of trusted participants agree to pool fixed periodic contributions (daily, weekly, or monthly) into a collective fund. At each scheduled cycle, the accumulated sum is distributed as a payout to one selected participant according to predetermined rules (such as a lottery draw, seniority ranking, or mutual auction).

Despite handling hundreds of millions of Ethiopian Birr (ETB) annually across urban and rural demographics, traditional Equb operations rely overwhelmingly on manual bookkeeping: paper registers, physical notebooks, handwritten receipts, and unorganized SMS/Telegram chat logs. This reliance on informal manual administration causes frequent bookkeeping errors, misplaced records, payment status disputes, lack of transparent member oversight, and vulnerability to unilateral misappropriation by organizers.

The **EthioEqub Digital Management System** is a professional, secure, role-based web application engineered to digitally manage and govern Equb group operations end-to-end. The system digitizes membership onboarding, automated contribution scheduling, manual payment recording with transaction reference tracking, dual-control payout authorization workflows, dispute management, transparent audit logging, and downloadable financial statements.

> **Core Financial Boundary & Integrity Principle**
>
> In Version 1.0 (v1), the EthioEqub system **does not hold custody of user funds nor directly execute automated bank debits**. Money continues to flow through established external channels (cash, commercial bank transfers, Telebirr, and CBE Birr). The system serves as the definitive, immutable digital ledger of record that reconciles and audits all commitments, receipts, payouts, and adjustments against verifiable transaction references. All simulated or test data is strictly isolated and clearly flagged.

3. Background and Problem Statement

3.1 Societal & Cultural Background

In Ethiopia, informal financial institutions—chief among them Equb (savings/credit) and Iddir (bereavement/insurance)—predate the formal banking system and maintain deep cultural legitimacy. For millions of micro-entrepreneurs, civil servants, merchants, and diaspora families, Equb provides vital access to lump-sum capital without the collateral hurdles, bureaucratic paperwork, and high interest rates associated with conventional commercial banks.

Equb arrangements range from small family circles (5–10 members contributing 500 ETB weekly) to massive commercial market associations in Addis Ababa (50–100+ merchants contributing 20,000–100,000 ETB daily or weekly). Equb operations are typically managed by a leadership group consisting of a Chairman/Organizer (**Sebsabi / ሰብሳቢ**), a Treasurer (**Genzeb Yazhi / ገንዘብ ያዥ**), and a Secretary (**Tsehafi / ጸሐፊ**).

3.2 Problem Analysis

Through field observations and stakeholder interviews with Equb organizers and participants, five core systemic problems were identified:

- **Vulnerability and Physical Loss of Records:** Physical ledgers deteriorate, become water-damaged, or get lost during changes of leadership. If a notebook is misplaced, entire multi-month payment histories and outstanding debt balances are obliterated.
- **Payment and Balance Disputes:** Members frequently assert they paid via cash or bank transfer on a specific date, while the treasurer's notebook indicates a pending or delinquent balance. Without a centralized, verifiable reference lookup, disputes can destabilize the group or dissolve friendships.
- **Unilateral Control and Lack of Dual Control:** In traditional setups, a single treasurer decides when payouts are disbursed, creating opportunities for delayed payments, favoritism, or outright embezzlement without independent counter-signatures.
- **Bookkeeping Overhead and Calculation Inaccuracies:** Organizers spend hours manually tallying who paid, who owes late fees, who has already collected their payout, and how much remains in the collective pool. Fractional errors and miscalculations multiply across cycles.
- **Lack of Real-Time Member Transparency:** Ordinary members have no visibility into the current pool status, other members' compliance rates, or upcoming payout queues unless they attend physically or request verbal updates.

3.3 Formal Problem Statement

Problem Statement

Traditional Equb associations in Ethiopia lack a secure, accessible, and verifiable digital management system that preserves the cultural ethos of community savings while eliminating bookkeeping errors, record loss, unauthorized payout disbursements, and payment disputes through automated scheduling, dual-approval controls, and an immutable audit trail.

4. Project Objectives and Expected Benefits

4.1 General Objective

To design, engineer, verify, and document a secure, production-ready, full-stack digital management system that automates the administration, financial recordkeeping, payout governance, and auditing of Ethiopian Equb associations.

4.2 Specific Objectives

1. **Group & Rule Configuration:** Enable Equb organizers to configure groups with customizable frequencies (daily, weekly, monthly), contribution amounts, member counts, and payout rules.
2. **Automated Cycle Scheduling:** Automatically generate mathematically consistent contribution cycles and payout schedules upon group inception.
3. **Immutable Financial Recording:** Provide treasurers with a streamlined interface to record member payments (cash, bank transfer, Telebirr, CBE Birr) with required transaction reference numbers and zero in-place ledger deletions.
4. **Dual-Control Payout Governance:** Implement a mandatory two-person approval workflow (Treasurer + Secondary Approver) before a payout is recognized as disbursed.
5. **Real-Time Member Self-Service:** Provide members with a responsive dashboard displaying their personal payment history, outstanding balances, upcoming deadlines, and payout queue position.
6. **Comprehensive Financial Reporting:** Generate downloadable periodic reconciliation statements (CSV/PDF) reflecting total collected funds, disbursed payouts, and outstanding arrears.
7. **Tamper-Evident Audit Logging:** Capture a permanent, append-only log of every financial creation, status modification, dispute, and administrative action, including actor identity, timestamp, and IP address.
8. **Pedagogical Excellence:** Structure the codebase and documentation to serve as an exemplary model of clean software engineering for a student developer, adhering to strict phase-by-phase verification.

4.3 Expected Benefits Matrix

Stakeholder	Traditional Pain Point	Expected Benefit in EthioEqub
Equb Members	Uncertainty regarding payment status, lost receipts, lack of payout queue transparency.	Immediate access to personal statement, verified receipt references, transparent payout queue.
Treasurers & Organizers	Hours spent on manual bookkeeping, balancing ledgers, tracing uncredited bank transfers.	Automated balance calculations, fast contribution entry with duplicate reference checks, instant CSV/PDF export.
Equb Group as a Whole	Disputes leading to group dissolution, suspicion of embezzlement, delayed payouts.	Dual-control approval prevents unauthorized withdrawals; immutable audit trail resolves disputes impartially.
Software Student	Lack of real-world experience in full-stack architecture, financial integrity, and security.	Deep mastery of professional software engineering practices, TypeScript, PostgreSQL, and security design.

5. Project Scope and Limitations

5.1 In-Scope Capabilities (Version 1.0)

- **User Authentication & Profiles:** Email/phone registration, password hashing (Argon2id/bcrypt), role-based authorization, session management via HttpOnly cookies.
- **Group & Membership Lifecycle:** Creation of groups, rule specification, member invitations, role assignments (Admin, Treasurer, Manager, Member), and member deactivation.
- **Cycle & Schedule Engine:** Automated calculation of contribution deadlines, cycle states, and payout dates based on frequency and start date.
- **Financial Ledger Operations:** Manual entry of contributions with payment channel categorization (Cash, Telebirr, CBE Birr, Bank Transfer, Other) and unique reference logging.
- **Adjustment Accounting:** Prohibiting hard deletions; correcting data entry mistakes via signed adjustment entries with audit reasons.
- **Dual-Control Payouts:** Two-step approval requirement for disbursing pooled funds to cycle recipients.
- **Dispute Management:** Mechanism for members to flag a disputed transaction with an admin resolution workflow.
- **Dashboards & Analytics:** Visual statistics on pool collection, delinquent members, upcoming cycles, and payout progress.
- **Reporting Engine:** Exportable CSV ledger tables and downloadable PDF account statements.
- **Notification Infrastructure:** Email notification dispatch for due dates and receipts, with an architectural abstraction ready for SMS integration.

5.2 Out-of-Scope Capabilities (Deferred to Future Versions)

- **Direct Payment Gateway Settlement:** The system will not process live automated debit/credit card charges or direct Telebirr/CBE API API merchant wallet withdrawals in v1.0. (Reason: Requires commercial financial licenses, banking merchant contracts, and capital reserves).
- **Escrow / Fund Custody:** The software platform never holds legal custody of user funds.
- **Native Mobile Applications:** No native Swift (iOS) or Kotlin (Android) apps. The frontend is built as a fully responsive Progressive Web Application (PWA) compatible with mobile browsers.
- **Multi-Currency Cross-Border Forex:** Default currency is Ethiopian Birr (ETB). Multi-currency conversions are deferred.
- **Micro-Lending / Interest-Bearing Loans:** Traditional Equb is zero-interest. Secondary auction bidding rules will be supported structurally in logic, but complex compound interest calculations are out of scope.

5.3 Explicit Project Constraints

Operational Constraints

- **Connectivity:** Target users operate on varying mobile internet connectivity. API payloads must remain lightweight (< 50 KB) and optimized for low latency.
- **Cost Constraint:** As an academic final-year software engineering project, all infrastructure must operate seamlessly on zero-cost tiers (Render, Neon, Vercel, Supabase) without requiring paid cloud subscriptions.
- **SMS Notification Fallback:** Ethiopian SMS gateway aggregators (e.g., Afromessage) require registered business licenses. In development and academic demonstration, Email and simulated in-app notifications serve as the primary verified delivery channel.

6. Stakeholders and User Roles

6.1 Stakeholder Classification

Stakeholder Group	Description & Interest	Primary Expectations
Academic Faculty	University Computer Science Examination Board	Robust software architecture, clean code, rigorous testing, security best practices, and thorough documentation.
Equb Organizers (Sebsabi)	Community leaders who create and oversee Equb groups	Simple group setup, transparent member tracking, and dispute-free operations.
Equb Treasurers (Genzeb Yazhi)	Individuals entrusted with recording collections and payouts	Fast data entry, automated balance calculations, clear reference checks, and verifiable receipts.
Equb Members	Participants contributing savings and awaiting payouts	Mobile accessibility, real-time balance transparency, and assurance that their payments are properly credited.
External Auditors / Elders	Community dispute arbitrators and independent reviewers	Unmodifiable, chronological audit logs of all group actions.

6.2 Role-Based Access Control (RBAC) Matrix

The system implements a granular, role-based permission hierarchy enforced on every backend API endpoint:

System Permission	System Admin	Group Owner	Treasurer	Manager	Member
Platform Configuration & Monitoring	YES	NO	NO	NO	NO
Create Equb Group & Define Rules	YES	YES	NO	NO	NO
Invite Members & Assign Roles	YES	YES	NO	YES	NO
Record Member Contribution	YES	YES	YES	NO	NO
Create Financial Adjustment Entry	YES	YES	YES	NO	NO
Primary Payout Approval (Approver 1)	YES	YES	YES	NO	NO
Secondary Payout Approval (Approver 2)	YES	YES	NO	YES	NO
Raise Transaction Dispute	NO	NO	NO	NO	YES
Resolve Dispute & Add Findings	YES	YES	NO	NO	NO
View Group Financial Reports & CSV	YES	YES	YES	YES	OWN ONLY
Inspect System & Group Audit Logs	YES	YES	NO	NO	NO

7. Functional Requirements

Functional requirements define the specific behaviors, actions, calculations, and services the system must execute. Each requirement is uniquely identified and prioritized using the MoSCoW framework (Must, Should, Could, Won't).

ID	Requirement Description	Role	Priority	Acceptance Criteria
FR-001	User Registration, Login, and Password Recovery.	Public / User	Must	User can sign up with name, email, phone, and password. Passwords must pass strength validation and be hashed. Login yields a secure HttpOnly cookie session.
FR-002	Equb Group Creation and Parameter Configuration.	Group Owner	Must	Owner can create a group with title, description, contribution amount (minor units), currency (ETB), frequency (daily/weekly/monthly), total member count, and start date.
FR-003	Automated Contribution and Cycle Schedule Generation.	System Engine	Must	When a group is initialized, the system automatically computes all cycle dates, deadlines, and payout dates matching the defined frequency and member count.
FR-004	Member Invitation and Role Assignment.	Group Owner / Manager	Must	Admins can invite registered users to join a group, designate their specific group role (Treasurer, Manager, Member), and assign their initial payout position.
FR-005	Manual Contribution Recording with Transaction Proof.	Treasurer / Owner	Must	Treasurer can record an incoming payment specifying member ID, cycle ID, amount, payment method (Cash, Bank, Telebirr, CBE Birr), date, and unique reference number.
FR-006	Real-Time Outstanding Balance and Arrears Computation.	System Engine	Must	The system continuously calculates total expected vs. total received per member per cycle, flagging statuses: `PENDING`, `PAID`, `PARTIAL`, or `LATE`.
FR-007	Payout Queue and Payout Generation.	System Engine	Must	System maps each cycle to exactly one beneficiary member based on group rules (lottery draw, seniority order, or approved bid). Total payout equals total pool collection.
FR-008	Dual-Control Payout Authorization Workflow.	Treasurer & Owner/Manager	Must	Payout disbursement requires two distinct approvals from authorized accounts. Neither approver can act as both Approver 1 and Approver 2 on the same payout.
FR-009	Member Self-Service Dashboard & Account Statement.	Member	Must	Members can log in to view their payment history, current balance, upcoming cycle deadlines, payout position, and download their official PDF statement.

ID	Requirement Description	Role	Priority	Acceptance Criteria
FR-010	Exportable Financial Reports (CSV & PDF).	Treasurer / Owner	Must	Generate structured CSV and PDF exports for period contributions, member balances, payout disbursements, and general group ledger reconciliation.
FR-011	Immutable Audit Trail of Sensitive Operations.	System Engine	Must	Every financial insertion, status update, adjustment, dispute, and approval automatically records an unmodifiable row capturing actor ID, timestamp, IP, and state diff.
FR-012	Automated Contribution Reminders.	System Engine	Should	Asynchronous job triggers automated reminder emails to members 48 hours and 24 hours prior to a cycle contribution deadline.
FR-013	Group Freeze and Emergency Deactivation.	Group Owner / Admin	Should	Authorized admin can freeze an Equb group in the event of legal disputes or member default, preventing further entries while retaining full read-only history.
FR-014	Contribution Dispute Filing and Resolution.	Member & Admin	Should	A member can submit a dispute on an inaccurate contribution record. Admin reviews evidence, resolves with a formal note, and triggers an adjustment entry.
FR-015	SMS Reminder Dispatch via Gateway Adapter.	System Engine	Could	Integrate SMS provider adapter (e.g., Afromessage) for mobile text notifications, configurable via external credentials.
FR-016	Bilingual User Interface (Amharic & English).	UI Layer	Could	Externalize UI strings through an i18n localization dictionary, allowing instant toggle between English and Amharic (አማርኛ).
FR-017	Visual Dashboard KPIs and Collection Progress.	Owner / Treasurer	Must	Display interactive visual metrics: Total Capital Collected, Total Payouts Disbursed, Collection Rate %, and Arrears Distribution.
FR-018	Financial Adjustment Accounting (Zero Hard Deletion).	Treasurer / Owner	Must	Mistaken entries are never overwritten or deleted. Adjustments are logged as separate debit/credit differential rows linking back to the parent contribution.
FR-019	Duplicate Reference Detection and Blocking.	System Engine	Must	Prevent accidental duplicate entries by enforcing unique database constraints on payment references within the same group and payment channel.

ID	Requirement Description	Role	Priority	Acceptance Criteria
FR-020	Two-Factor Authentication (TOTP) for Privileged Roles.	Treasurer / Admin	Should	Provide optional TOTP-based 2FA (Google Authenticator) for users holding Treasurer and Owner roles prior to authorizing payouts.

8. Non-Functional Requirements

Non-functional requirements (NFRs) specify the qualitative, architectural, security, and operational standards the system must satisfy.

ID	Category	Specification & Standard	Verification Method
NFR-001	Cryptographic Security	Passwords hashed using Argon2id or bcrypt (minimum 12 salt rounds). Session tokens stored in secure, HttpOnly, SameSite=Lax cookies. All transmission over TLS 1.3 / HTTPS.	Security code audit & OWASP dependency check.
NFR-002	Strict Server-Side AuthZ	Zero trust on client input. Every protected API route enforces role-based access control (RBAC) and group ownership checks in server middleware.	Automated integration test suite with unauthorized requests.
NFR-003	API Performance	95% of standard API read queries respond within 500 ms under typical group loads (up to 100 members). Database queries properly indexed.	Load benchmarking with Artillery / Autocannon.
NFR-004	Architectural Scalability	Stateless backend server architecture allowing independent horizontal scaling. Database connection pooling configured via Prisma / PgBouncer.	Architecture review & stateless concurrency test.
NFR-005	Service Availability	System targets 99.0% availability during active operational cycles in v1.0. Graceful error degradation upon third-party outage.	Uptime monitoring (UptimeRobot / Healthcheck endpoint).
NFR-006	Transactional Reliability	All multi-entity financial operations (contributions, adjustments, payouts) wrapped in PostgreSQL atomic ACID transactions (`prisma.$transaction`). No partial writes.	Automated failure-injection transaction tests.
NFR-007	Immutable Auditability	Audit log is append-only. No API endpoint or database trigger permits `UPDATE` or `DELETE` on the `AuditLog` table. Retained for group lifetime.	Database permission verification & unit tests.
NFR-008	Mobile Usability	UI designed mobile-first using responsive layouts (Tailwind CSS). Touch targets ≥ 48px. Complies with WCAG 2.1 Level AA contrast standards.	Lighthouse accessibility audit (> 90 score).
NFR-009	Maintainability & Typing	Strict TypeScript on frontend and backend (`strict: true`). Zero `any` types in core financial modules. Shared Zod validation schemas.	Static type checking (`tsc --noEmit`) and ESLint.
NFR-010	Cloud Portability	Fully containerizable via standard Dockerfile. Deployable to modern cloud providers (Render, Railway, Fly.io, Vercel).	Docker container build and local run test.
NFR-011	Data Privacy & PII Protection	Minimization of personally identifiable information (PII). Secrets, API tokens, and database credentials stored exclusively in `.env`, never in version control.	Git-secrets scan and pre-commit verification.
NFR-012	Resilient Error Handling	Standardized JSON error envelope across all endpoints. Internal database exceptions, stack traces, and SQL queries suppressed in production.	Negative test assertions on API error responses.
NFR-013	Localization Readiness	All user-facing strings decoupled from JSX markup into structured language resource objects, enabling rapid Amharic translation.	Code review of UI localization dictionaries.

ID	Category	Specification & Standard	Verification Method
NFR-014	Automated Test Coverage	Minimum 80% automated test coverage across core financial calculation algorithms, schedule generators, and authorization middleware.	Vitest coverage report output.
NFR-015	System Observability	Structured JSON logging with request correlation IDs (`x-request-id`) to trace transactions across client, server, and database layers.	Log inspection in staging environment.

9. Business Rules for Equb Operations

Business rules define the operational policies, constraints, and financial calculation invariants of the Equb domain. Every rule must be strictly enforced in the backend domain layer.

BR-001: Integer Minor-Unit Currency Representation

All monetary amounts must be stored and computed as **64-bit integers in minor units** (cents / santim). For Ethiopian Birr (ETB), 1 Birr = 100 santim. (e.g., 5,000 ETB is represented as `500000`). Floating-point data types (`FLOAT`, `DOUBLE`) are strictly prohibited for financial balances to prevent IEEE-754 precision rounding errors.

BR-002: Cycle Count and Member Synchronization

The number of cycles in an Equb group equals the number of full membership shares (N). Each full member is entitled to receive exactly one payout during the complete Equb round. If a member purchases a fractional share (e.g., 0.5 share), they must be paired with another fractional member to constitute one full payout cycle, or receive half the pooled payout.

BR-003: Contribution Lifecycle & Late Payment State Engine

A contribution record follows an explicit state transition machine:
`PENDING` → `PAID` (full amount received on/before due date)
`PENDING` → `PARTIAL` (partial amount received; balance remains pending)
`PENDING` → `LATE` (due date has elapsed without required contribution)
`PAID / PARTIAL` → `DISPUTED` (member challenges the recorded record)

BR-004: Payment Method Classification & Mandatory Reference

Every contribution record must specify an authorized payment channel: `CASH`, `BANK_TRANSFER`, `TELEBIRR`, `CBE_BIRR`, or `OTHER`. For all electronic channels (Bank, Telebirr, CBE Birr), a unique external transaction reference number (e.g., Telebirr transaction ID or CBE slip number) is mandatory. Cash transactions must record the treasurer's physical receipt book number.

BR-005: Duplicate Reference Collision Prevention

The system shall enforce a unique composite constraint on `(group_id, payment_method, reference_number)` for all non-null references. Any attempt to record a contribution using an already recorded reference within that group shall be rejected immediately with an HTTP 409 Conflict.

BR-006: Immutable Ledger & Adjustment Accounting

Under no circumstances may an existing financial contribution row be edited or deleted (no in-place `UPDATE` or `DELETE`). If a treasurer records an erroneous amount or assigns it to the wrong member, the correction must be effected via an `Adjustment` record specifying the delta (positive or negative), the correction rationale, and the authorized actor ID.

BR-007: Dual-Control Payout Authorization

Disbursement of a cycle payout pool requires independent authorization from two distinct privileged users: **Approver 1** (typically the Treasurer verifying fund availability) and **Approver 2** (the Group Owner or Manager confirming member eligibility). The system shall reject any approval where `Approver 1 ID == Approver 2 ID`. *Phase 2 Decision:* For small informal or family groups (< 5 members), the Group Owner may configure single-approver disbursement at group creation if enabled in group bylaws.

BR-008: Payout Sequencing Algorithms

The order of payouts is established at group formation through one of three transparent algorithms:

1. **Pre-Agreed Seniority (JOIN_ORDER):** Sequential schedule based on agreed roster or initial join date.
2. **Verifiable Lottery Draw (LOTTERY):** Randomized selection executed at cycle inception, recorded in the audit log.
3. **Auction / Bidding (AUCTION):** In groups that permit bidding, members bid a discount rebate to claim an urgent payout. The discount is distributed as a dividend among remaining members.

BR-009: Group Dissolution & Freezing Rules

An Equb group cannot be deleted while active cycles or outstanding balances exist. An owner may only mark a group as `FROZEN` (suspending all writes during dispute resolution) or `CLOSED` (upon completion of all cycles and full zero-balance reconciliation). Historical data remains permanently accessible read-only.

BR-010: Dispute Freezing and Resolution Protocol

When a member raises a dispute against a recorded contribution:

- The contribution status immediately transitions to `DISPUTED`.
- The disputed amount is withheld from aggregate payout pool calculations until resolved.
- Resolution requires an Administrator or Owner to submit a formal investigation finding, triggering either an adjustment entry or reaffirmation of the original record.

BR-011: Strict Non-Negative Balance & Overpayment Rules

Overpayments by members are permitted and credited to an unallocated prepaid balance. In subsequent cycles, the prepaid balance is automatically drawn down to satisfy new contribution demands. Negative contributions are strictly forbidden.

BR-012: Complete Audit Context Capture

Every mutating API request on a financial entity must extract and persist: (1) Actor User ID, (2) Client IP Address, (3) User-Agent Header, (4) Precise UTC Timestamp, (5) Action Type, and (6) Pre- and Post-Mutation JSON state diffs.

BR-013: Member Replacement and Default Guarantee (Wasa)

If a member defaults after having already received their payout, the traditional Equb rule requires their assigned guarantor (**Wasa / ዋስ**) to assume contribution liability. *Phase 2 Decision:* Guarantor assignment is configurable per group: strictly mandatory for members receiving early payouts if enabled in group bylaws (protecting the collective pool post-disbursement), and optional for other cycles.

BR-014: Cycle Pool Solvency Invariant

A payout cannot be released if the total confirmed collected contributions for that cycle do not equal the required pool sum, unless the group explicitly authorizes an advance loan from group reserve funds, which must be audited as a reserve draw.

10. User Journeys and Use Cases

10.1 Use Case Summary

UC ID	Use Case Title	Primary Actor	Core Objective
UC-01	Configure Equb & Generate Schedule	Group Owner	Establish a new Equb group, invite members, and generate all cycle deadlines.
UC-02	Record Member Contribution	Treasurer	Record manual payment receipt with unique reference number and verify balance.
UC-03	Dual Payout Authorization	Treasurer & Owner	Authorize and confirm payout disbursement to the designated cycle beneficiary.
UC-04	Self-Service Statement & Balance	Member	View live status, check payout position, and download verified PDF statement.
UC-05	Submit & Resolve Dispute	Member & Admin	Flag an inaccurate payment entry, provide proof, and review administrative finding.
UC-06	Period Reconciliation & Export	Treasurer	Reconcile group ledger against external bank statements and export CSV/PDF reports.

10.2 Detailed Use Case Specifications

UC-01: Configure Equb Group and Generate Schedule

- **Actor:** Group Owner (Sebsabi)
- **Pre-conditions:** Owner is authenticated and holds verified email credentials.
- **Main Flow:**
  1. Owner navigates to "Create New Equb" screen.
  2. Owner enters Title ("Bole Commercial Equb Round 3"), Description, Contribution Amount (10,000 ETB), Frequency (Weekly), Number of Members (20), Start Date (2026-11-01), and Payout Rule (JOIN_ORDER).
  3. System validates inputs (amount > 0, members >= 2, date >= today).
  4. System renders an interactive preview of all 20 calculated cycles with projected due dates and payout dates.
  5. Owner confirms creation. System executes an atomic transaction creating the `Group` and 20 `Cycle` records.
  6. Owner invites 20 members via email/phone link.
- **Post-conditions:** Group status set to `ACTIVE`; schedule persisted; invitations queued.

UC-02: Record Member Contribution with Reference Proof

- **Actor:** Treasurer (Genzeb Yazhi)

- **Pre-conditions:** Group is active; cycle is in `OPEN` state; member is registered in group.
- **Main Flow:**
  1. Treasurer selects current cycle and chooses Member from roster.
  2. System displays member's current expected balance (e.g., 10,000 ETB).
  3. Treasurer inputs received amount (10,000 ETB), payment method (`TELEBIRR`), and reference number (`TB-9847291048`).
  4. System checks database for reference uniqueness within the group.
  5. Treasurer clicks "Submit Contribution".
  6. Backend validates payload with Zod, checks role authorization, persists `Contribution` row inside a transaction, and writes an unmodifiable `AuditLog` row.
  7. Dashboard updates instantly; an automated email receipt is dispatched to the member.
- **Alternative Flow (Duplicate Reference):** If reference `TB-9847291048` exists, system rejects write with HTTP 409 and displays error: "Transaction reference already recorded for this group on 2026-10-15."

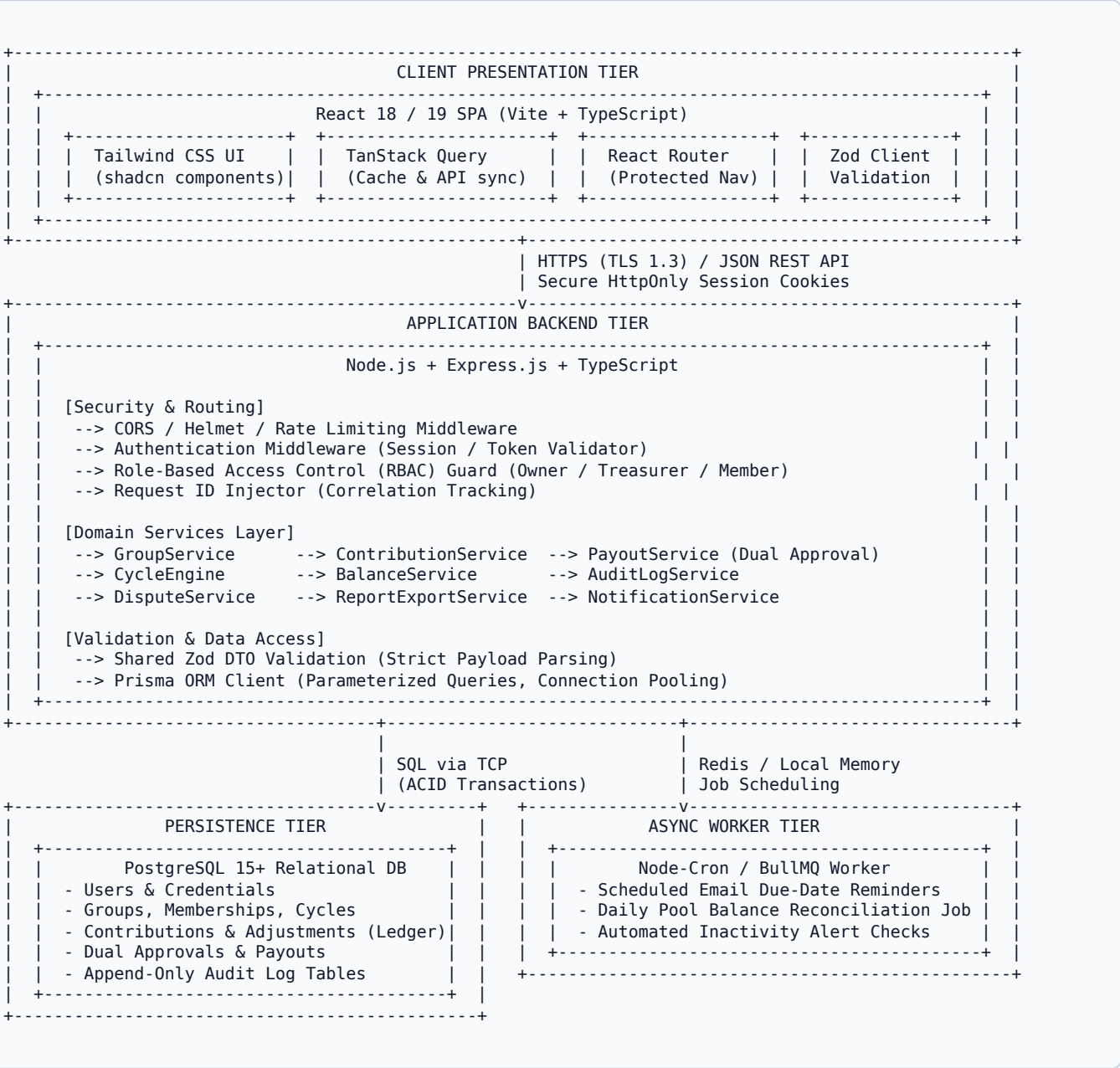
UC-03: Dual-Control Payout Authorization and Issuance

- **Actors:** Approver 1 (Treasurer) and Approver 2 (Group Owner / Manager)
- **Pre-conditions:** Cycle collection deadline has arrived; total pool is verified; payout recipient is identified.
- **Main Flow:**
  1. Treasurer opens "Payout Approvals" screen, reviews Cycle 4 recipient (Abebe Kebede, Pool: 200,000 ETB).
  2. Treasurer confirms all 20 member contributions are reconciled and clicks "Authorize Payout (Approver 1)".
  3. System updates Payout record status to `PENDING_SECOND_APPROVAL`, recording Approver 1 user ID and timestamp.
  4. Group Owner receives an approval notification, logs into dashboard from their separate device.
  5. Group Owner reviews payout details and clicks "Finalize Disbursement (Approver 2)".
  6. System verifies `Approver 1 != Approver 2`, marks payout `PAID`, logs disbursement date, and updates group pool ledger.
- **Post-conditions:** Recipient notified; payout marked complete; audit log permanently updated.

11. System Architecture and Component Responsibilities

11.1 Architectural Pattern

The system follows a modern **3-Tier Layered Architecture with a Modular Monolith backend**. This architecture was selected deliberately to provide clean separation of concerns, strong type-safety, rapid development velocity, and operational simplicity suitable for academic defense and production reliability.



11.2 Component Responsibilities

Component	Core Responsibilities	Design Decisions & Guarantees
Frontend Client (React)	Renders accessible, responsive dashboard views; manages client-side form state; provides optimistic UI updates; formats minor-unit currency into user-friendly ETB displays.	Built with Vite for sub-second hot module reload. Uses Tailwind CSS for mobile-first responsiveness. Client validation is treated as UX enhancement only; never trusted for security.
API Gateway / Router	Terminates HTTP requests; validates SSL/TLS headers; applies IP rate-limiting; parses JSON payloads; verifies session cookies.	Strict CORS origin whitelisting; Helmet security headers; centralized error handling middleware preventing server stack trace leaks.
Service Layer (Express)	Executes business logic: cycle calculation algorithms, dual-control payout verification, duplicate reference prevention, balance reconciliation.	Clean architecture: controllers only handle HTTP transport, delegating all domain rules to decoupled, unit-testable service classes.
Persistence Layer (Prisma & Postgres)	Enforces relational integrity, foreign key cascading constraints, unique compound indexes, and ACID guarantees for financial writes.	PostgreSQL selected for transactional durability. Prisma provides end-to-end TypeScript type inference from database schema to frontend DTOs.
Background Worker	Executes time-based tasks: sending reminder notifications before deadlines, identifying late members, and calculating daily audit totals.	Runs as an isolated scheduled process; deduplicates notification dispatches using idempotency keys.

12. Technology Stack and Justification

12.1 Technology Selection Matrix

Layer	Selected Technology	Evaluated Alternatives	Engineering Justification
Frontend Core	**React 18/19 + Vite + TypeScript**	Next.js, Vue 3, Angular	Single-Page Application (SPA) architecture allows lightning-fast navigation, zero server-rendering overhead, simple deployment on free CDNs (Vercel/Netlify), and strong component reusability.
CSS & UI System	**Tailwind CSS + Lucide Icons**	Bootstrap, Material UI, Chakra UI	Utility-first CSS produces minimal bundle sizes, complete design freedom, mobile-first responsiveness out-of-the-box, and eliminates bloated third-party CSS dependencies.
Backend API	**Node.js (LTS v20+) + Express + TypeScript**	Python (Django/FastAPI), Java (Spring Boot), Go	Uniform language (TypeScript) across the entire stack simplifies development for a student. Massive package ecosystem, lightweight footprint, and non-blocking I/O ideal for REST APIs.
Database	**PostgreSQL 15+**	MongoDB, MySQL, SQLite	Financial ledgers demand strict ACID transactional guarantees, strong foreign key constraints, and relational integrity. MongoDB's eventual consistency is unsuitable for financial ledgers.
ORM & Migrations	**Prisma ORM**	TypeORM, Drizzle, Sequelize	Declarative schema modeling, automatic migration history tracking, and unmatched TypeScript type safety. Prevents SQL injection via parameterized queries.
Schema Validation	**Zod**	Joi, Yup, Class-Validator	Enables static type inference directly from validation schemas. Allows sharing validation schemas between backend API controllers and frontend forms.
Authentication	**HttpOnly Secure Cookies + Argon2id / bcrypt**	LocalStorage JWT, OAuth-only	Storing authentication tokens in `httpOnly, Secure, SameSite=Lax` cookies completely mitigates Cross-Site Scripting (XSS) token theft vulnerabilities.

Layer	Selected Technology	Evaluated Alternatives	Engineering Justification
Automated Testing	**Vitest + Supertest**	Jest, Mocha/Chai	Vitest runs in the native Vite pipeline, providing 10x faster execution than legacy Jest, native ESM support, and built-in TypeScript execution.
Version Control & CI	**Git + GitHub + GitHub Actions**	GitLab, Bitbucket	Industry standard. Automated GitHub Actions run type-checks, linters, and test suites on every pull request.
Hosting Strategy	**Vercel (SPA) + Render (API) + Neon (Postgres)**	AWS (ECS/RDS), DigitalOcean	Generous free tiers perfectly tailored for academic defense demonstrations without ongoing operational costs. Fully production-capable.

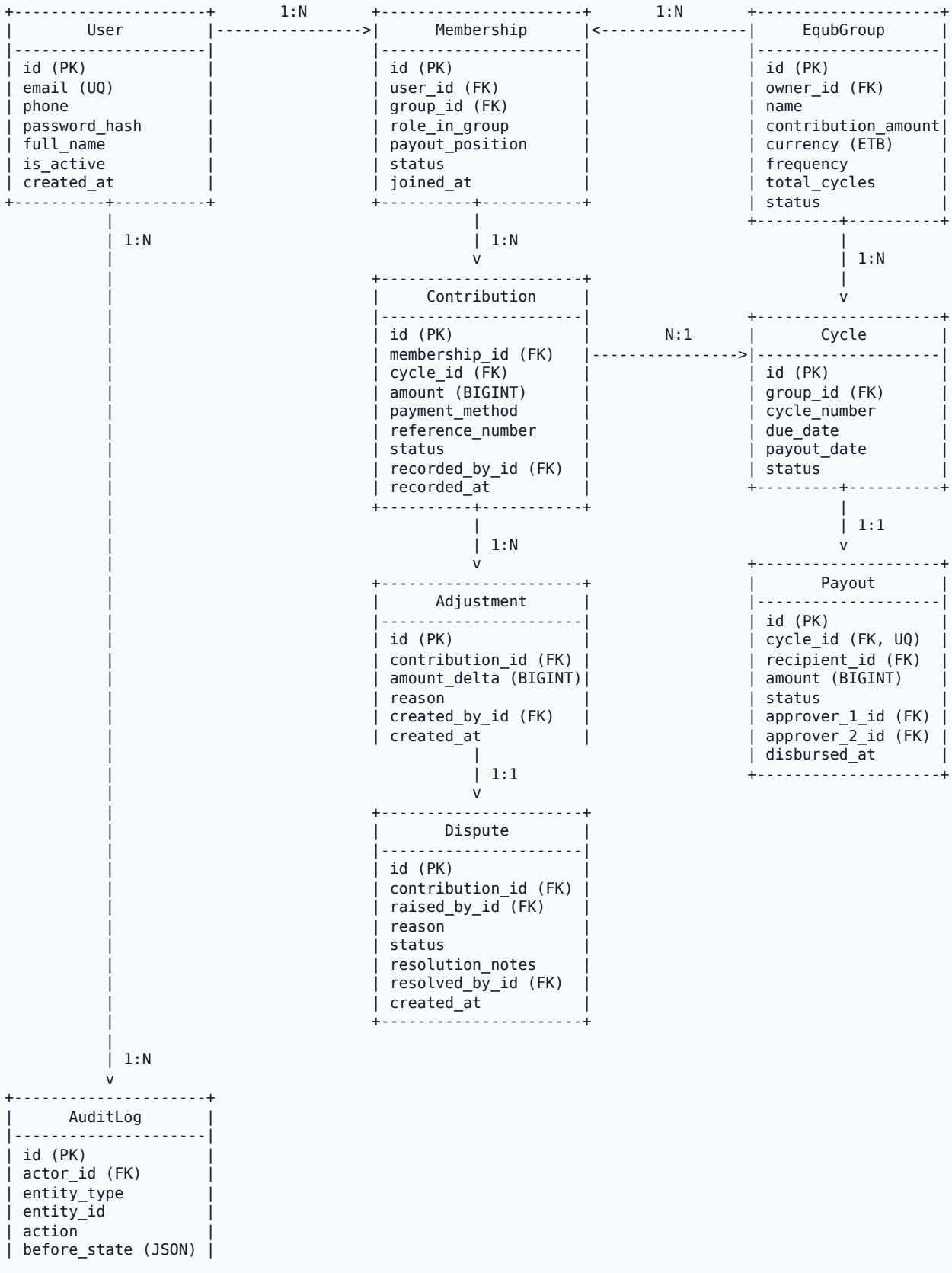
12.2 Prerequisites for Developer Environment

- **Runtime Environment:** Node.js version 20.x or higher (LTS).
- **Package Manager:** npm version 10.x or higher.
- **Version Control:** Git version 2.40+ configured with GitHub.
- **Database Server:** PostgreSQL 15+ (local instance or cloud connection string from Neon.tech / Supabase).
- **Code Editor:** Visual Studio Code with recommended extensions: ESLint, Prettier, Prisma, Tailwind CSS IntelliSense.

13. Database Design, Entities, Relationships, and ER

Diagram

13.1 Entity Relationship Diagram (ERD)



```
| after_state (JSON)  |
| ip_address          |
| user_agent          |
| created_at          |
+---------------------+
```

13.2 Database Schema Specifications

Table: User

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique user identifier.
email	VARCHAR(255)	UNIQUE, NOT NULL	Normalized lower-case email address.
phone	VARCHAR(32)	NOT NULL	Ethiopian mobile phone (+251...).
password_hash	VARCHAR(255)	NOT NULL	Argon2id or bcrypt hash.
full_name	VARCHAR(128)	NOT NULL	User's legal full name.
is_active	BOOLEAN	DEFAULT TRUE	Account active flag.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Registration timestamp.

Table: EqubGroup

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique group identifier.
owner_id	UUID	FOREIGN KEY → User(id), ON DELETE RESTRICT	Creator/Admin of the Equb group.
name	VARCHAR(128)	NOT NULL	Group title (e.g. Merkato Textile Equb).
description	TEXT	NULL	Detailed purpose and group bylaws.
contribution_amount	BIGINT	NOT NULL, CHECK (> 0)	Amount in minor units (santim).
currency	VARCHAR(3)	DEFAULT 'ETB'	ISO currency code.
frequency	VARCHAR(16)	NOT NULL, CHECK (daily, weekly, monthly)	Contribution frequency interval.
total_cycles	INTEGER	NOT NULL, CHECK (>= 2)	Total planned cycles (= member count).
payout_rule	VARCHAR(32)	DEFAULT 'JOIN_ORDER'	LOTTERY, JOIN_ORDER, or AUCTION.
status	VARCHAR(16)	DEFAULT 'ACTIVE'	ACTIVE, FROZEN, or COMPLETED.
start_date	DATE	NOT NULL	Official commencement date.

Table: Membership

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique membership identifier.
user_id	UUID	FOREIGN KEY → User(id), ON DELETE RESTRICT	Member user account.
group_id	UUID	FOREIGN KEY → EqubGroup(id), ON DELETE RESTRICT	Equb group ID.
role_in_group	VARCHAR(16)	DEFAULT 'MEMBER'	OWNER, TREASURER, MANAGER, MEMBER.
payout_position	INTEGER	CHECK (> 0)	Assigned cycle for payout receipt.
status	VARCHAR(16)	DEFAULT 'ACTIVE'	ACTIVE, SUSPENDED, COMPLETED.
joined_at	TIMESTAMPTZ	DEFAULT NOW()	Date member joined group.

Constraint: UNIQUE (user_id, group_id) — prevents duplicate memberships.

Table: Cycle

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique cycle identifier.
group_id	UUID	FOREIGN KEY → EqubGroup(id), ON DELETE CASCADE	Associated Equb group.
cycle_number	INTEGER	NOT NULL	Sequential round number (1..N).
due_date	DATE	NOT NULL	Deadline for member contributions.
payout_date	DATE	NOT NULL	Scheduled payout disbursement date.
status	VARCHAR(16)	DEFAULT 'PENDING'	PENDING, OPEN, RECONCILED, CLOSED.

Constraint: UNIQUE (group_id, cycle_number) — prevents duplicate cycle indices.

Table: Contribution

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique contribution record ID.
membership_id	UUID	FOREIGN KEY → Membership(id), ON DELETE RESTRICT	Contributing member.
cycle_id	UUID	FOREIGN KEY → Cycle(id), ON DELETE RESTRICT	Target contribution cycle.
amount	BIGINT	NOT NULL, CHECK (> 0)	Recorded payment in minor units.
payment_method	VARCHAR(32)	NOT NULL	CASH, TELEBIRR, CBE_BIRR, BANK, OTHER.
reference_number	VARCHAR(128)	NULL	External payment receipt or TX code.
status	VARCHAR(16)	DEFAULT 'PAID'	PAID, PARTIAL, DISPUTED, ADJUSTED.
recorded_by_id	UUID	FOREIGN KEY → User(id), ON DELETE RESTRICT	Acting Treasurer / Admin ID.
recorded_at	TIMESTAMPTZ	DEFAULT NOW()	Exact entry timestamp.

Index: UNIQUE (membership_id, cycle_id, reference_number) WHERE reference_number IS NOT NULL.

Table: Adjustment

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique adjustment record ID.
contribution_id	UUID	FOREIGN KEY → Contribution(id), ON DELETE RESTRICT	Parent erroneous contribution.
amount_delta	BIGINT	NOT NULL	Signed difference in minor units (+/-).
reason	TEXT	NOT NULL	Auditable justification for change.
created_by_id	UUID	FOREIGN KEY → User(id), ON DELETE RESTRICT	Authorizing Treasurer / Admin.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp of adjustment.

Table: Payout

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique payout ID.
cycle_id	UUID	UNIQUE, FOREIGN KEY → Cycle(id), ON DELETE RESTRICT	One payout per cycle.
recipient_id	UUID	FOREIGN KEY → Membership(id), ON DELETE RESTRICT	Beneficiary member.
amount	BIGINT	NOT NULL, CHECK (> 0)	Total pooled amount disbursed.
status	VARCHAR(24)	DEFAULT 'SCHEDULED'	SCHEDULED, PENDING_APPROVAL, APPROVED, DISBURSED.
approver_1_id	UUID	NULL, FOREIGN KEY → User(id)	First dual-control approver (Treasurer).
approver_2_id	UUID	NULL, FOREIGN KEY → User(id)	Second dual-control approver (Admin).
disbursed_at	TIMESTAMPTZ	NULL	Actual disbursement confirmation time.

Constraint: CHECK (approver_1_id != approver_2_id) — enforces dual control.

Table: AuditLog (Append-Only)

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique audit log sequence ID.
actor_id	UUID	NULL, FOREIGN KEY → User(id)	Acting user who executed action.
entity_type	VARCHAR(32)	NOT NULL	User, Group, Contribution, Payout, etc.
entity_id	UUID	NOT NULL	Target record UUID.
action	VARCHAR(32)	NOT NULL	INSERT, UPDATE_STATUS, ADJUST, APPROVE.
before_state	JSONB	NULL	Snapshot of data prior to operation.
after_state	JSONB	NULL	Snapshot of data post operation.
ip_address	VARCHAR(45)	NULL	IPv4 or IPv6 client address.
user_agent	TEXT	NULL	Browser / device identification.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Immutable creation timestamp.

14. API Design and Representative Endpoints

14.1 REST API Standards & Envelope Conventions

The EthioEqub REST API conforms to strict RESTful conventions, using semantic HTTP verbs, predictable resource paths prefixed with `/api/v1`, and standardized JSON responses.

Success Response Envelope

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "requestId": "req_88f91a0c4e",
    "timestamp": "2026-10-24T06:15:30.120Z",
    "pagination": {
      "page": 1,
      "pageSize": 20,
      "totalRecords": 85,
      "totalPages": 5
    }
  }
}
```

Error Response Envelope

```
{
  "success": false,
  "error": {
    "code": "DUPLICATE_REFERENCE",
    "message": "Payment reference 'TB-9847291048' has already been recorded in this group.",
    "details": [
      { "field": "referenceNumber", "issue": "Must be unique within group" }
    ],
    "requestId": "req_88f91a0c4e"
  }
}
```

14.2 Representative API Endpoint Inventory

Method	Endpoint Path	Purpose	Minimum Role	Success Code
`POST`	/api/v1/auth/register	Register new user account	Public	201 Created
`POST`	/api/v1/auth/login	Authenticate and set session cookie	Public	200 OK
`POST`	/api/v1/auth/logout	Terminate session and clear cookie	Authenticated	200 OK
`GET`	/api/v1/users/me	Fetch authenticated profile	Authenticated	200 OK
`POST`	/api/v1/groups	Create group and generate cycles	Authenticated	201 Created
`GET`	/api/v1/groups/:id	Get group overview and stats	Member	200 OK
`POST`	/api/v1/groups/:id/members	Invite / add member to group	Group Owner	201 Created
`GET`	/api/v1/groups/:id/cycles	List cycles and schedule status	Member	200 OK
`POST`	/api/v1/groups/:id/contributions	Record member contribution	Treasurer	201 Created
`POST`	/api/v1/contributions/:id/adjustments	Post an auditable adjustment	Treasurer	201 Created
`POST`	/api/v1/payouts/:id/approve	Submit dual-control approval	Treasurer / Owner	200 OK
`POST`	/api/v1/contributions/:id/disputes	File a dispute on a record	Member	201 Created
`GET`	/api/v1/groups/:id/reports/ledger.csv	Export group CSV transaction ledger	Treasurer	200 OK
`GET`	/api/v1/groups/:id/audit-logs	Inspect immutable audit records	Group Owner	200 OK

14.3 Sample API Request and Response Walkthroughs

Endpoint: POST /api/v1/groups/:id/contributions

Request Headers: `Content-Type: application/json`, `Cookie: session_id=...`

```
// POST Payload
{
  "membershipId": "c4b12f67-8e9a-4d22-92b0-817a5e8c1b22",
  "cycleId": "e1f82c33-912b-4566-b33a-112233445566",
  "amountMinorUnits": 1000000,          // 10,000.00 ETB
  "paymentMethod": "TELEBIRR",
  "referenceNumber": "TB-9847291048",
  "notes": "Paid at Bole branch retail agent"
}
```

```
// 201 Created Response
{
  "success": true,
  "data": {
    "contributionId": "7a8b9c0d-1e2f-3a4b-5c6d-7e8f9a0b1c2d",
    "memberFullName": "Abebe Kebede",
    "cycleNumber": 4,
    "amountFormatted": "10,000.00 ETB",
    "paymentMethod": "TELEBIRR",
    "referenceNumber": "TB-9847291048",
    "recordedAt": "2026-10-24T06:20:00Z",
    "status": "PAID",
    "remainingMemberBalanceMinorUnits": 0
  },
  "meta": { "requestId": "req_88f91a0c4e" }
}
```

15. UI/UX Design Plan and Screen Inventory

15.1 Design Philosophy & Visual Principles

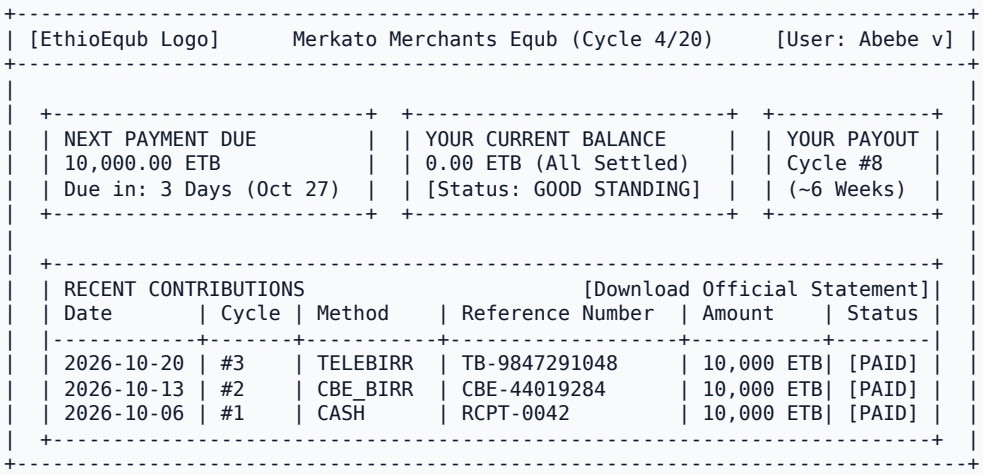
- **Clarity Over Decoration:** Financial interfaces must convey unambiguous trust, precise decimal alignment, and clear status badges.
- **Mobile-First Usability:** Over 85% of Ethiopian Equb participants access internet services via smartphones. All member journeys are designed for single-hand mobile operation (touch targets ≥ 48px).
- **High Contrast & Accessibility:** Complies with WCAG 2.1 AA contrast requirements to ensure legibility in bright sunlight.
- **Zero Ambiguity Financial Badges:** Every financial state is visually distinct: `LATE / DELINQUENT` `PENDING / DUE` `CONFIRMED / PAID`

15.2 Screen Inventory (S-01 through S-13)

ID	Screen Title	Target Audience	Key UI Components & Capabilities
S-01	Authentication & Password Reset	All Users	Clean card layout, tabs for Login/Register, password strength indicator, accessible error toasts.
S-02	Member Personal Dashboard	Equb Member	Next due date alert card, balance overview, upcoming payout countdown, recent payment history table.
S-03	Group Executive Dashboard	Owner / Treasurer	KPI metric tiles (Total Collected, Delinquent Amount, Solvency Ratio), cycle progress bar, fast-action toolbar.
S-04	Group Configuration & Rules	Group Owner	Multi-step wizard: Name, frequency, contribution amount, cycle schedule preview, rule toggles.
S-05	Member Roster & Roles	Owner / Manager	Searchable table with member roles, payout positions, invitation dispatch modal, member status toggles.
S-06	Contribution Recording Terminal	Treasurer	Optimized fast-entry form: Member autocomplete, amount prefill, payment method tabs, reference validator.
S-07	Member Contribution History	Member / Treasurer	Chronological ledger breakdown per cycle, reference verification links, dispute button, PDF download.
S-08	Payout Schedule & Rotation	All Group Users	Visual timeline showing cycle numbers, scheduled payout dates, beneficiary member names, and completion badges.
S-09	Dual Payout Approval Gateway	Treasurer / Owner	Dual signature modal showing Approver 1 and Approver 2 status badges, pool solvency check, authorization button.
S-10	Financial Reporting & Exports	Treasurer / Owner	Date-range filter, breakdown by payment method, one-click CSV export and print-ready PDF balance sheet.
S-11	Immutable Audit Log Explorer	Group Owner / Auditor	Filterable log table: Timestamp, Actor, Action, Entity, before/after JSON visual diff modal.
S-12	Dispute Resolution Center	Member / Owner	Filing form with file attachment simulation, admin resolution notes, and trigger adjustment action.
S-13	Profile & Security Settings	All Users	Contact update, password modification, session activity log, and optional TOTP 2FA setup.

15.3 Core Wireframe Layout Concepts

Wireframe: S-02 Member Dashboard



16. Authentication, Authorization, Privacy, Security, and Audit Requirements

16.1 Threat Modeling & Countermeasure Framework

Threat Vector	Potential Exploitation / Impact	Implemented Countermeasure
Credential Stuffing & Brute Force	Automated scripts attempting dictionary passwords against user logins.	Express rate-limiting (maximum 5 failed login attempts per 15 minutes per IP); strong password entropy requirements; Argon2id hashing.
Cross-Site Scripting (XSS)	Malicious scripts injected via notes stealing auth tokens from LocalStorage.	Auth tokens stored strictly in `HttpOnly, Secure, SameSite=Lax` cookies inaccessible to JavaScript; React auto-escaping; Helmet CSP headers.
Cross-Site Request Forgery (CSRF)	Forged requests triggering unauthorized contribution or payout actions.	SameSite cookie flags; custom Anti-CSRF header verification (`x-requested-with`) on all state-changing mutating verbs.
Broken Object-Level AuthZ (BOLA / IDOR)	User altering URL parameter (e.g., `/groups/999`) to view or edit another group.	Middleware verifies that authenticated user possesses explicit active Membership in target group for every request.
SQL Injection	Unsanitized search fields altering backend SQL statements.	Prisma ORM generates 100% parameterized SQL queries; manual raw query strings prohibited in production code.
Ledger Tampering by Rogue Treasurer	Treasurer altering past contribution figures to conceal cash shortfalls.	Database tables enforce append-only policies; financial mutations executed strictly via audited Adjustment records; zero in-place updates.

16.2 Cryptographic & Session Specifications

- **Password Storage:** Salted and hashed using Argon2id with 64 MB memory cost, 3 iterations, and parallelism of 1 (fallback: bcrypt with 12 salt rounds).
- **Session Lifecycle:** Session tokens are cryptographically random 256-bit entropy strings with a 7-day expiration sliding window. Inactive sessions terminate after 24 hours.
- **Audit Log Immutability:** The `AuditLog` table has no application-level update or delete endpoints. A PostgreSQL trigger can optionally raise an exception if an `UPDATE` or `DELETE` SQL statement is attempted on this table.

17. Financial Transaction Integrity, Reconciliation, and Reporting

17.1 Mathematical Precision Standards

Financial applications must never suffer from IEEE-754 floating-point rounding errors (where `0.1 + 0.2 = 0.30000000000000004`). EthioEqub mandates:

- **Storage:** All currency amounts are stored as 64-bit signed integers (`BIGINT` in PostgreSQL) representing minor currency units (santim / cents).
- **Formulas:** $$\text{Stored Santim} = \text{ETB Amount} \times 100$$ $$\text{Displayed ETB} = \frac{\text{Stored Santim}}{100.00}$$
- **Arithmetic:** All additions, subtractions, and balance summations occur strictly on integer values. Division is restricted to final payout split distributions with explicit integer remainder allocation.

17.2 Transactional Atomicity via ACID Guarantees

Every financial event spanning multiple records (e.g., recording a contribution, updating the cycle running balance, and inserting an audit log entry) is wrapped in a PostgreSQL atomic transaction:

```
// Atomic Financial Execution Pattern (Prisma Transaction)
await prisma.$transaction(async (tx) => {
  // 1. Verify group is active and not frozen
  const group = await tx.equbGroup.findUniqueOrThrow({ where: { id: groupId } });
  if (group.status !== 'ACTIVE') throw new DomainError('GROUP_FROZEN');

  // 2. Prevent duplicate reference within group
  if (referenceNumber) {
    const existing = await tx.contribution.findFirst({
      where: { groupId, paymentMethod, referenceNumber }
    });
    if (existing) throw new ConflictError('DUPLICATE_PAYMENT_REFERENCE');
  }

  // 3. Persist contribution record
  const contribution = await tx.contribution.create({
    data: { membershipId, cycleId, amountMinorUnits, paymentMethod, referenceNumber, recordedById }
  });

  // 4. Persist immutable audit log entry
  await tx.auditLog.create({
    data: {
      actorId: recordedById,
      entityType: 'Contribution',
      entityId: contribution.id,
      action: 'RECORD_CONTRIBUTION',
      afterState: contribution,
      ipAddress, userAgent
    }
  });

  return contribution;
});
```

17.3 Automated Pool Reconciliation Invariant

At the conclusion of every cycle, the system validates the fundamental Equb conservation law:

> **The Equb Invariant Equation**
>
> $$\sum_{i=1}^{M} \text{Contributions}_c(i) + \sum \text{Adjustments}_c = \text{Payout}_c + \text{Carried Reserve}_c$$
> Where M is total group membership and c is the current cycle index. If the calculated delta does not equal zero, the system flags a reconciliation variance, preventing cycle closure until rectified by the Treasurer.

18. Notification and Payment Integration Strategy

18.1 Payment Reality in Ethiopia

In contemporary Ethiopia, informal savings groups transact across four primary channels:

1. **Telebirr (Ethio Telecom):** The dominant mobile money service in Ethiopia, ubiquitous among urban and rural merchants.
2. **CBE Birr & Commercial Bank of Ethiopia (CBE):** Mobile banking apps and branch account-to-account transfers.
3. **Private Bank Mobile Apps:** Awash Bank, Dashen Bank (Amole), Bank of Abyssinia.
4. **Physical Cash:** In-person cash handoffs during traditional weekly coffee meetings.

18.2 Version 1.0 vs. Future Roadmap Architecture

Dimension	Version 1.0 Architecture (Current Scope)	Version 2.0 Roadmap (Future Evolution)
Payment Execution	Manual entry by Treasurer with mandatory transaction reference capture. Zero direct custody or fund movement.	Direct API integration with Telebirr Merchant Gateway and CBE Open Banking API for automated webhook verification.
Verification Model	Dual reconciliation: Treasurer verifies SMS receipt on physical phone, inputs reference code into EthioEqub ledger.	Server-to-server webhook confirmation instantly reconciling payments without manual intervention.
Simulated Data Rule	Any demo transactions, seeded members, or test records must carry explicit `is_simulated: true` flags and UI badges.	Strict sandbox test environments partitioned from live production ledgers.
Notifications	Transactional Email (Resend / SendGrid) + in-app notification center. SMS provider decoupled via adapter interface.	Direct SMS text dispatch via Afromessage / Ethio Telecom SMS aggregator API.

19. Validation Rules and Error-Handling Strategy

19.1 Centralized Zod Schema Validation

Every incoming API payload is parsed and strictly validated against strongly typed Zod schemas prior to reaching domain controllers. Unrecognized fields are stripped (`strip()`) to prevent parameter injection.

```
// Example Zod Schema: Group Creation DTO
export const CreateGroupSchema = z.object({
  name: z.string().min(3, "Group name must be at least 3 characters").max(100),
  description: z.string().max(500).optional(),
  contributionAmountMinorUnits: z.number().int().positive("Amount must be a positive integer"),
  frequency: z.enum(["DAILY", "WEEKLY", "MONTHLY"]),
  totalCycles: z.number().int().min(2, "Equb requires at least 2 cycles/members").max(100),
  startDate: z.string().datetime().refine((date) => new Date(date) >= new Date(), {
    message: "Start date must be today or in the future"
  }),
  payoutRule: z.enum(["JOIN_ORDER", "LOTTERY", "AUCTION"]).default("JOIN_ORDER")
});
```

19.2 Standardized HTTP Status Code Mapping

HTTP Code	Error Code	Trigger Condition & Meaning
400 Bad Request	VALIDATION_ERROR	Request payload violates Zod schema (e.g. negative amount, invalid email).
401 Unauthorized	AUTH_REQUIRED	Missing, expired, or invalid session cookie.
403 Forbidden	FORBIDDEN_ROLE	User lacks necessary role permissions (e.g., Member attempting to record payment).
404 Not Found	RESOURCE_NOT_FOUND	Specified group, cycle, or user ID does not exist in the database.
409 Conflict	DUPLICATE_REFERENCE	Payment reference already recorded within group.
422 Unprocessable	BUSINESS_RULE_VIOLATION	Violates Equb invariant (e.g., Approver 1 attempting to act as Approver 2).
500 Internal Error	INTERNAL_SERVER_ERROR	Unhandled server exception; stack trace suppressed in production response.

20. Testing Strategy and Acceptance Criteria

20.1 Test Pyramid Methodology

- **Unit Tests (60%):** Isolated verification of domain business logic: cycle calculation algorithms, minor-unit currency conversions, late balance determination, and Zod validation rules.
- **Integration Tests (30%):** Testing API endpoints against a dedicated PostgreSQL test container: testing authentication flows, database transactions, dual-control payout constraints, and duplicate reference rejections.
- **End-to-End (E2E) Sanity Tests (10%):** Full user journey verification using Playwright: User signs up → Creates Group → Records Contribution → Approves Payout → Exports CSV.

20.2 Formal Acceptance Criteria Register

ID	Test Scenario	Expected Verification Result
AC-01	Group Creation Schedule Integrity	Creating a group with 10 members and weekly frequency generates exactly 10 cycles, sequentially dated 7 days apart without gaps.
AC-02	Duplicate Payment Reference Rejection	Submitting a second contribution with identical (group, paymentMethod, referenceNumber) is rejected with HTTP 409 Conflict.
AC-03	Dual-Control Payout Rejection on Self-Approval	Attempting to submit both Approver 1 and Approver 2 from the same authenticated user ID fails with HTTP 422 Business Rule Violation.
AC-04	Tenant Isolation on Member Statements	Member A querying `/api/v1/users/me/contributions` receives only their personal payment records and cannot access Member B's data.
AC-05	Atomic Rollback on Transaction Failure	If an error occurs while writing an AuditLog row during contribution creation, the entire database transaction aborts; zero orphaned records created.
AC-06	Adjustment Audit Preservation	Submitting an adjustment modifies the net computed balance but leaves the original contribution row intact with an associated adjustment history.
AC-07	CSV Ledger Export Hygiene	Exported CSV ledger files encode all strings safely to prevent CSV formula injection attacks (prepending single quotes to `=, +, -, @`).
AC-08	Session Revocation on Logout	Calling `POST /api/v1/auth/logout` invalidates the session server-side; subsequent requests using the old cookie return HTTP 401 Unauthorized.

21. Development Phases and Milestones

To ensure professional engineering quality, disciplined progress, and thorough student learning, the project follows 12 strictly sequenced phases. Progression to each subsequent phase requires explicit stakeholder confirmation via the command `next`.

Phase	Phase Name & Focus	Key Deliverables & Verification Checkpoints	Status
Phase 1	Project Documentation & Architecture Baseline	Publish complete 26-section PRTDD document, compile print-ready PDF, verify completeness.	**CURRENT PHASE**
Phase 2	Requirements Review & Decision Finalization	Review PRTDD with user, resolve open architectural questions (2FA, Amharic i18n, hosting).	Pending `next`
Phase 3	Environment Inspection & Prerequisite Verification	Inspect developer workstation (Node, npm, Git, PostgreSQL), execute safe read-only diagnostics.	Pending `next`
Phase 4	Project Skeleton & Structure Creation	Generate approved root directory structure, initial Git repo, and module boundaries.	Pending `next`
Phase 5	Architecture & Folder Walkthrough	Detailed explanation of every folder, configuration file, and architectural pattern for student.	Pending `next`
Phase 6	App & Database Configuration	Set up environment variables, TypeScript configs, ESLint/Prettier, Prisma client, DB connection.	Pending `next`
Phase 7	UI/UX Incremental Implementation	Build responsive Tailwind components, layout shells, dashboard cards, navigation bars.	Pending `next`
Phase 8	Database Schema, Migrations & Seeds	Execute Prisma migrations, verify PostgreSQL tables and constraints, seed realistic demo data.	Pending `next`
Phase 9	Authentication & RBAC Implementation	Implement registration, login, Argon2id hashing, secure cookie sessions, role middleware.	Pending `next`
Phase 10	Core Business Features Implementation	Build Equb group management, contribution recording, dual-control payouts, adjustments, reports.	Pending `next`
Phase 11	Automated Testing & QA Verification	Run Vitest unit tests, Supertest API integration tests, and manual verification checklist.	Pending `next`
Phase 12	Security Audit, Deployment & Final Defense	Execute security review, deploy API to cloud, connect live database, prepare defense demo.	Pending `next`

22. Risks, Assumptions, and Mitigation Strategies

22.1 Risk Assessment Matrix

Identified Risk	Severity	Likelihood	Mitigation Strategy
Scope Creep into Live Banking	HIGH	HIGH	Strictly enforce v1 boundary: EthioEqub is a verifiable record and audit ledger, not a licensed payment gateway or money transmitter.
Floating-Point Precision Loss	CRITICAL	MEDIUM	Mandate integer minor units (santim) across all database columns, backend models, and calculations. Enforce via unit tests.
SMS Gateway Account Rejection	MEDIUM	HIGH	Implement adapter design pattern: Transactional email (Resend) acts as primary verified notification engine; SMS adapter is pluggable.
Student Overwhelm / Knowledge Gaps	MEDIUM	MEDIUM	Enforce 10-step pedagogical delivery format: provide exact commands, explain all technical concepts, and verify before moving forward.
Rogue Ledger Modifications	HIGH	LOW	Prohibit in-place SQL updates and deletes on financial tables. Require adjustment rows and enforce append-only audit logging.

22.2 Core Assumptions & Finalized Stakeholder Decisions

Verified Assumptions:

- The developer has access to a modern workstation with Linux, macOS, or Windows WSL2.
- Cloud deployment will utilize reliable free-tier services (Render, Vercel, Neon PostgreSQL).
- Primary target users possess basic smartphone literacy and access to Telegram or web browsers.

Finalized Stakeholder Decisions (Phase 2 Baseline Sign-Off):

Agreed Architecture & Policy Baseline

1. **UI Language Strategy:** English-first UI with a clean, decoupled i18n localization dictionary architecture. Strings are externalized so full Amharic (አማርኛ) translation can be activated seamlessly in Phase 10 without JSX refactoring.
2. **Dual-Control Payout Governance:** Mandatory dual-control approval (Treasurer + Owner/Manager) for standard groups. For small informal or family groups (< 5 members), the Group Owner may configure single-approver payout disbursement at group creation.
3. **Guarantor (Wasa / ዋስ) Requirement:** Configurable per group in bylaws: strictly mandatory for members receiving early payouts (to ensure ongoing post-payout savings discipline), and optional for other cycles.
4. **Notification Strategy:** Transactional Email (Resend / SMTP) and an in-app Notification Center for development and academic defense demonstration, backed by a pluggable SMS adapter ready for live Ethiopian SMS gateway credentials.

23. Deployment, Backup, Recovery, and Maintenance Plan

23.1 Cloud Deployment Architecture

- **Client SPA:** Hosted on Vercel or Netlify global edge network, providing automatic SSL certificate issuance, instant cache invalidation, and seamless continuous deployment from GitHub.
- **Backend API:** Hosted as a managed Node.js container service on Render.com or Railway.app with environment-based configuration, automated health monitoring, and auto-restart capabilities.
- **Database Server:** Managed PostgreSQL on Neon.tech or Supabase, providing serverless connection pooling, point-in-time recovery, and SSL encryption in transit.

23.2 Backup and Disaster Recovery Protocol

Backup Type	Frequency	Retention	Storage Target
Automated Managed Snapshots	Daily (02:00 UTC)	7 Rolling Days	Neon / Supabase Managed Cloud Storage
Logical Database Dump (`pg_dump`)	Weekly (Sunday)	30 Days	Encrypted Private Object Storage (AWS S3)

Disaster Recovery Targets

Recovery Point Objective (RPO): Maximum 24 hours of data loss in catastrophic provider failure.
Recovery Time Objective (RTO): System restored to full operation within 4 hours via automated migration scripts.

24. Installation Prerequisites and Developer Environment Setup

The following development toolchain will be verified through non-destructive read-only terminal commands in Phase 3 prior to generating application files:

- **Node.js (LTS v20+):** Verified via `node -v`
- **npm Package Manager (v10+):** Verified via `npm -v`
- **Git Version Control:** Verified via `git --version`
- **PostgreSQL Client / Service:** Verified via `psql --version` (or cloud connection string)
- **Web Browser:** Google Chrome, Mozilla Firefox, or Chromium with DevTools for debugging.

25. Final Project Acceptance Checklist

Prior to final submission and academic project defense, the project must satisfy every verification checkpoint in this acceptance register:

Verified	Category	Acceptance Criteria
[ ]	Requirements Baseline	PRTDD approved by student and stakeholders; all 26 sections documented.
[ ]	Functional Completeness	All 20 Functional Requirements (FR-001 to FR-020) implemented and verified.
[ ]	Financial Integrity	All balances computed in integer minor units; zero float errors; pool invariant satisfied.
[ ]	Ledger Immutability	Zero in-place updates/deletions on financial rows; corrections via signed adjustment records.
[ ]	Dual-Control Payouts	Disbursements require two distinct authorized approvers; self-approval strictly rejected.
[ ]	Audit Log Completeness	100% of financial operations produce immutable audit rows capturing actor, timestamp, IP.
[ ]	Automated Testing	Unit and integration test suites pass with > 80% coverage on financial modules.
[ ]	Security & Secrets Hygiene	Zero secrets in Git history; password hashed with Argon2id; secure HttpOnly cookies.
[ ]	Report Generation	CSV ledger exports and PDF member statements download with correct totals.
[ ]	Live Cloud Deployment	System accessible on public URL with live PostgreSQL database and SSL encryption.

26. Glossary of Cultural and Technical Terms

Term	Definition and Context
Equb (ዕቁብ)	Traditional Ethiopian Rotating Savings and Credit Association (ROSCA) wherein members pool periodic fixed contributions and distribute the accumulated sum to one winner per cycle.
Sebsabi (ሰብሳቢ)	The elected chairman, organizer, or administrative owner of an Equb group responsible for governance, rules, and dispute oversight.
Genzeb Yazhi (ገንዘብ ያዥ)	The trusted treasurer of the Equb group responsible for collecting member payments, maintaining the ledger, and issuing receipts.
Wasa (ዋስ)	A legal or community guarantor who pledges financial liability if a member defaults on their post-payout contribution obligations.
Cycle (ዙር)	A single scheduled operational iteration (daily, weekly, or monthly) consisting of contribution collection and one payout disbursement.
Payout (ዕጣ)	The aggregated sum of member contributions collected during a cycle and awarded to the designated beneficiary.
Minor Units	The smallest non-divisible fractional denomination of a currency (e.g. santim for Ethiopian Birr; 1 ETB = 100 santim). Stored as integers to prevent floating-point calculation errors.
ROSCA	Rotating Savings and Credit Association: an international peer-to-peer financial model recognized globally.
Dual Control	A mandatory security governance policy requiring two independent authorized individuals to sign off before a sensitive financial action can execute.
ACID	Atomicity, Consistency, Isolation, Durability: database guarantees ensuring that financial transactions are processed reliably without partial state corruption.
RBAC	Role-Based Access Control: an authorization mechanism restricting system access based on assigned user roles (Admin, Treasurer, Manager, Member).
Argon2id	The premier modern password hashing algorithm, resistant to GPU/ASIC cracking and side-channel timing attacks.
Idempotency	An API property ensuring that submitting the same request multiple times produces the identical outcome without duplicate transactions.
Zod	A TypeScript-first schema declaration and runtime data validation library.
Prisma	A next-generation Object-Relational Mapper (ORM) for Node.js and TypeScript providing type-safe database queries and automated migrations.